

DOM Events — Complete Reference Guide

For developers who know DOM well and need clarity on **which event fires when, on what, and how they differ**.

1. Mouse Events

Event	When it Fires	Popular Use Case
<code>click</code>	Full press + release on same element	Buttons, links, toggles
<code>dblclick</code>	Two rapid clicks	Text selection, zoom, edit-in-place
<code>mousedown</code>	Button pressed (before release)	Drag start, custom right-click
<code>mouseup</code>	Button released	Drag end, drop actions
<code>mousemove</code>	Pointer moves over element	Drawing canvas, tooltips, parallax
<code>mouseenter</code>	Pointer enters element	Show tooltip/dropdown
<code>mouseleave</code>	Pointer leaves element	Hide tooltip/dropdown
<code>mouseover</code>	Pointer enters element or children	Hover highlight with delegation
<code>mouseout</code>	Pointer leaves element or children	Hover highlight with delegation
<code>contextmenu</code>	Right-click (before menu shows)	Custom right-click menus

Key Diffs

`mouseenter` **VS** `mouseover` :

- `mouseenter` **does NOT bubble**, fires only when entering the element itself.
- `mouseover` **bubbles**, fires again when entering child elements.

- Use `mouseenter/leave` for simple hover. Use `mouseover/out` when you need event delegation.

```
// mouseover fires multiple times as you move between children
parent.addEventListener('mouseover', () => console.log('over')); // fires on every
parent.addEventListener('mouseenter', () => console.log('enter')); // fires ONCE
```

click VS mousedown + mouseup :

- `click` = full cycle (down → up on same element). Won't fire if you mousedown on A and mouseup on B.
- `mousedown` fires instantly on press — better for drag operations where you need immediate response.

2. Keyboard Events

Event	When it Fires	Popular Use Case
<code>keydown</code>	Key pressed (repeats if held)	Shortcuts, game controls, form nav
<code>keyup</code>	Key released	Search-on-type (after user stops input)
<code>keypress</code>	 DEPRECATED — don't use	—

Key Diffs

keydown VS keyup :

- `keydown` fires **immediately** and **repeats** if key is held. Best for shortcuts (`Ctrl+S`).
- `keyup` fires **once** on release. Best for “do something after user finishes pressing”.

```
// Ctrl+S save shortcut — use keydown
document.addEventListener('keydown', (e) => {
  if (e.ctrlKey && e.key === 's') {
    e.preventDefault();
    save();
  }
});
```

```
// Search after typing — use keyup
searchInput.addEventListener('keyup', (e) => {
```

```
fetchResults(e.target.value);  
});
```

 **keypress is deprecated.** It doesn't fire for modifier keys (Shift, Ctrl, Alt) and has inconsistent behavior. Always use `keydown` or `keyup`.

 **Works on:** Any focusable element, but commonly attached to `document`, `window`, or `<input>`.

3. Form / Input Events

Event	When it Fires	Popular Use Case
<code>input</code>	Value changes immediately (every keystroke/paste)	Live search, character counter
<code>change</code>	Value changes and element loses focus (on commit)	Dropdowns, checkboxes, final values
<code>focus</code>	Element gains focus	Show helper text, highlight field
<code>blur</code>	Element loses focus	Validate on leave, hide suggestions
<code>focusin</code>	Same as focus but bubbles	Event delegation for focus
<code>focusout</code>	Same as blur but bubbles	Event delegation for blur
<code>submit</code>	Form is submitted	Validate before sending, SPA forms
<code>reset</code>	Form reset button clicked	Confirm reset, clear custom state
<code>select</code>	Text is selected inside input/textarea	Copy selection, show formatting bar
<code>invalid</code>	Input fails built-in validation on submit	Custom error messages

Key Diffs

`input` **vs** `change` — **THE most confusing pair:**

Scenario	input fires?	change fires?
Typing in <code><input type="text"></code>	✅ Every keystroke	✅ On blur only
Pasting text	✅ Immediately	✅ On blur only
<code><input type="checkbox"></code> click	✅ Immediately	✅ Immediately
<code><input type="range"></code> drag	✅ Every movement	✅ On release only
<code><select></code> option pick	✅ Immediately	✅ Immediately
Programmatic <code>el.value = 'x'</code>	❌ No	❌ No

```
// Live character counter – use input
textarea.addEventListener('input', (e) => {
  counter.textContent = e.target.value.length;
});

// Save to DB only when user is done – use change
textarea.addEventListener('change', (e) => {
  saveToDB(e.target.value);
});
```

focus/blur VS focusin/focusout :

- focus/blur **do NOT bubble**. Can't use event delegation.
- focusin/focusout **bubble**. Use these when attaching to a parent.

```
// Won't work for dynamically added inputs
form.addEventListener('focus', handler); // ❌ doesn't bubble

// Works with delegation
form.addEventListener('focusin', handler); // ✅ bubbles
```

🎯 **submit** and **reset** fire ONLY on `<form>` elements, not on buttons.

🎯 **select** works ONLY on `<input>` and `<textarea>`.

🎯 **invalid** works ONLY on form elements with validation attributes (`required`, `pattern`, etc).

4. Clipboard Events

Event	When it Fires	Popular Use Case
copy	User copies (Ctrl+C)	Track copying, modify copied text
cut	User cuts (Ctrl+X)	Prevent cutting in readonly fields
paste	User pastes (Ctrl+V)	Sanitize pasted HTML, block paste

```
input.addEventListener('paste', (e) => {  
  const text = e.clipboardData.getData('text/plain');  
  // Sanitize or validate pasted content  
  if (text.includes('<script>')) e.preventDefault();  
});
```

 Works on any focusable element, most commonly `<input>`, `<textarea>`, and `contenteditable` elements.

5. Drag & Drop Events

Event	Fires On	When
dragstart	Dragged element	User starts dragging
drag	Dragged element	Continuously while dragging
dragend	Dragged element	Drag operation ends (drop or cancel)
dragenter	Drop target	Dragged item enters a valid drop zone
dragover	Drop target	Dragged item is over drop zone (repeats)
dragleave	Drop target	Dragged item leaves drop zone
drop	Drop target	Item is dropped on valid target

```
// MUST preventDefault on dragover, otherwise drop won't fire  
dropZone.addEventListener('dragover', (e) => e.preventDefault());  
dropZone.addEventListener('drop', (e) => {  
  e.preventDefault();  
});
```

```
const data = e.dataTransfer.getData('text/plain');
});
```

 **Requires** `draggable="true"` attribute on the element being dragged (except `<img>` and `<a>` which are draggable by default).

6. Touch Events (Mobile Only)

Event	When it Fires	Popular Use Case
<code>touchstart</code>	Finger touches screen	Start gesture tracking
<code>touchmove</code>	Finger moves on screen	Swipe, pinch-to-zoom
<code>touchend</code>	Finger lifts off	Complete gesture, tap
<code>touchcancel</code>	Touch interrupted (call/alert)	Cleanup gesture state

Key Diff: Touch vs Mouse

- Touch events fire **before** mouse events (300ms delay for click on mobile).
- **A single tap fires:** `touchstart` → `touchend` → `mousemove` → `mousedown` → `mouseup` → `click`.
- Use `touch` events for gesture-heavy mobile UIs. For simple clicks, `click` works on both.

 **Mobile/tablet only.** Desktop browsers don't fire these (unless using dev tools emulation).

7. Scroll & Wheel Events

Event	When it Fires	Popular Use Case
<code>scroll</code>	Element or page scrolls	Infinite scroll, sticky nav, lazy load
<code>wheel</code>	Mouse wheel rotates (before scroll)	Custom scroll behavior, zoom canvas

Key Diff

- `scroll` **fires after** scroll happens. Can't prevent it.
- `wheel` **fires before** scroll. Can `preventDefault()` to block native scroll.

```
// Infinite scroll - use scroll
window.addEventListener('scroll', () => {
  if (window.innerHeight + window.scrollY >= document.body.offsetHeight - 100) {
    loadMore();
  }
});

// Custom zoom on canvas - use wheel
canvas.addEventListener('wheel', (e) => {
  e.preventDefault(); // block page scroll
  zoomLevel += e.deltaY > 0 ? -0.1 : 0.1;
}, { passive: false });
```

 **Performance tip:** Always use `{ passive: true }` for scroll listeners unless you **NEED** `preventDefault()`. Browsers optimize passive listeners.

8. Window / Document Lifecycle Events

Event	Fires On	When	Popular Use Case
<code>DOMContentLoaded</code>	<code>document</code>	HTML parsed, DOM ready (before images/CSS)	Init scripts, DOM manipulation
<code>load</code>	<code>window</code>	Everything loaded (images, styles, iframes)	Hide loader, start animations
<code>beforeunload</code>	<code>window</code>	User about to leave page	"Unsaved changes" warning
<code>unload</code>	<code>window</code>	Page is unloading	Analytics beacon, cleanup
<code>resize</code>	<code>window</code>	Browser window resized	Responsive layout recalculation
<code>hashchange</code>	<code>window</code>	URL hash (#) changes	SPA routing (older approach)
<code>popstate</code>	<code>window</code>	Browser back/forward button	SPA routing with

			History API
visibilitychange	document	Tab becomes visible/hidden	Pause video, reduce API calls

Key Diff

DOMContentLoaded VS load :

HTML parsed → DOMContentLoaded fires → images/css load → load fires

- **DOMContentLoaded : DOM is ready, but images might still be loading. Use this 99% of the time.**
- **load : Everything including images, stylesheets, iframes loaded. Use only when you need image dimensions etc.**

```
// Best for most init code
document.addEventListener('DOMContentLoaded', () => {
  initApp();
});
```

```
// Only when you need all resources
window.addEventListener('load', () => {
  hideLoadingSpinner();
});
```

🎯 DOMContentLoaded **fires on** document **only**. load **fires on** window, , <script>, <link>.

🎯 resize **fires on** window **only**. Debounce it — it fires rapidly.

🎯 visibilitychange — **check** document.visibilityState === 'hidden' to know the state.

9. Media Events

Event	When it Fires	Popular Use Case
play	Media starts playing	Update play/pause button
pause	Media pauses	Update UI, save position

ended	Media finishes playing	Autoplay next, show replay
timeupdate	Current time changes (frequent)	Progress bar, timestamp
volumechange	Volume or mute changes	Update volume slider
loadeddata	First frame loaded	Show video thumbnail
canplay	Enough data to start playing	Enable play button
waiting	Playback stopped due to buffering	Show buffering spinner
error	Media failed to load	Show fallback, retry

 **Works ONLY on** `<audio>` **and** `<video>` **elements.**

10. Transition & Animation Events (CSS)

Event	When it Fires	Popular Use Case
transitionstart	CSS transition begins	Add class during animation
transitionend	CSS transition completes	Remove element after fadeout
transitioncancel	CSS transition cancelled	Cleanup state
animationstart	CSS @keyframes animation begins	Track animation progress
animationend	CSS @keyframes animation ends	Remove/reset element
animationiteration	Animation loop repeats	Count loops, change behavior

```
// Remove element after fade-out transition
el.style.opacity = '0';
el.addEventListener('transitionend', () => el.remove(), { once: true });
```

Key Diff

- `transition*` events → for CSS `transition` property changes.
- `animation*` events → for CSS `@keyframes` animations.
- They are completely separate systems and fire independently.

11. Intersection & Resize Observer (Modern APIs)

These aren't traditional events but behave similarly and are frequently used:

API	What it Observes	Popular Use Case
<code>IntersectionObserver</code>	Element entering/leaving viewport	Lazy load images, infinite scroll, animate on scroll
<code>ResizeObserver</code>	Element size changes	Responsive components, charts
<code>MutationObserver</code>	DOM tree changes (add/remove/modify)	Dynamic content tracking, plugins

```
// Lazy load images – better than scroll event
const observer = new IntersectionObserver((entries) => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      entry.target.src = entry.target.dataset.src;
      observer.unobserve(entry.target);
    }
  });
});
document.querySelectorAll('img[data-src]').forEach(img => observer.observe(img));
```

12. Miscellaneous Events

Event	Fires On	When	Popular Use Case
<code>error</code>	<code></code> , <code><script></code> , window	Resource fails to load	Fallback image, error logging
<code>load</code>	<code></code> , <code><script></code> , <code><iframe></code>	Resource finishes loading	Lazy load confirm, iframe ready

abort	, <video>	Loading aborted	Cleanup, retry
-------	----------------	-----------------	----------------

Master Cheatsheet: Events That DON'T Bubble

Most events bubble, but these important ones **do NOT**:

Event	Bubbling Alternative
focus	focusin 
blur	focusout 
mouseenter	mouseover 
mouseleave	mouseout 
load	None
unload	None
scroll	None (but see note)
resize	None

 If you need event delegation, always pick the bubbling version.

Event Listener Options Quick Reference

```
element.addEventListener('click', handler, {
  capture: false, // fire during capture phase instead of bubble
  once: true,     // auto-remove after first fire
  passive: true,  // promise you won't call preventDefault (perf boost)
  signal: controller.signal // AbortController to remove listener
});
```